Analysis of Algorithms



CS311 - Design & Analysis of Algorithms

Dr. Maram Alajlan

majlan@psu.edu.sa

Algorithm

- An ***algorithm*** is a set of instructions to be followed to solve a problem.
  - There can be more than one solution (more than one algorithm) to solve a given problem.
- An algorithm must be correct. It should correctly solve the problem.

- Once we have a correct algorithm for a problem, we have to determine the efficiency of that algorithm.

Algorithmic Performance

There are *two aspects* of algorithmic performance:

- Time (more important in this course)
  - Instructions take time.
  - How fast does the algorithm perform?
  - What affects its runtime?
- Space
  - Data structures take space
  - What kind of data structures can be used?
  - How does choice of data structure affect the runtime?

❓ We will focus on time:

- How to estimate the time required for an algorithm
- How to reduce the time required

Analysis of Algorithms

- ***Analysis of Algorithms*** is the area of computer science that provides tools to analyze the efficiency of different methods of solutions.

- How do we compare the time efficiency of two algorithms that solve the same problem?

Naïve/experimental Approach: implement these algorithms in a programming language (C++, java, etc.), and run them to compare their time requirements.

Analysis of Algorithms

- 3-Sum: counts the number of triples in an array of n integers that sums to 0 https://algs4.cs.princeton.edu/14analysis/ThreeSum.java.html

```
public class ThreeSum
{
   public static int count(int[] a)
   {
      int N = a.length;
      int count = 0;
      for (int i = 0; i < N; i++)
         for (int j = i+1; j < N; j++)
            for (int k = j+1; k < N; k++)
               if (a[i] + a[j] + a[k] == 0)
                  count++;
      return count;
   }

   public static void main(String[] args)
   {
      In in = new In(args[0]);
      int[] a = in.readAllInts();
      StdOut.println(count(a));
   }
}
```

Measuring the running time

- Q. How to time a program?
- A. Manual

```
% java ThreeSum 1000 1Kints.txt
```



tick tick tick

```
70
% java ThreeSum 2000 2Kints.txt
```



tick tick tick tick tick tick tick tick
tick tick tick tick tick tick tick tick
tick tick tick tick tick tick tick tick

```
528
% java ThreeSum 4000 4Kints.txt
```



tick tick tick tick tick tick tick tick
tick tick tick tick tick tick tick tick
tick tick tick tick tick tick tick tick

Measuring the running time

- Q. How to time a program?
- A. Automatic

public class Stopwatch (part of stdlib.jar)	
Stopwatch()	*create a new stopwatch*
double elapsedTime()	*time since creation (in seconds)*

```
public static void main(String[] args)
{
   In in = new In(args[0]);
   int[] a = in.readAllInts();
   Stopwatch stopwatch = new Stopwatch();
   StdOut.println(ThreeSum.count(a));
   double time = stopwatch.elapsedTime();
   StdOut.println("elapsed time " + time);
}
```

Empirical analysis

- Run the program for various input sizes and measure running time.

Run it multiple times

N	time (seconds) †
250	0.0
500	0.0
1,000	0.1
2,000	0.8
4,000	6.4
8,000	51.1
16,000	?

Analysis of Algorithms

- Comparing the programs (instead of algorithms) has difficulties.
  - How are the algorithms coded?
    - Comparing running times means comparing the implementations.
    - We should not compare implementations, because they are sensitive to programming style that may cloud the issue of which algorithm is inherently more efficient.
  - What computer should we use?
    - We should compare the efficiency of the algorithms independently of a particular computer.
  - What data should the program use?
    - Any analysis must be independent of specific data.

- When we analyze algorithms, we should employ mathematical techniques that analyze algorithms independently of *specific implementations, computers, or data.*

- To analyze algorithms:
  - First, we start to count the number of significant operations in a particular solution to assess its efficiency.
  - Then, we will express the efficiency of algorithms using growth functions.

The Execution Time of Algorithms

- Each operation in an algorithm (or a program) has a cost.
  - Each operation takes a certain of time.

```
count = count + 1;
```

take a certain amount of time, but it is constant

A sequence of operations:

```
count = count + 1;
sum = sum + count;
```

Cost: c_1

Cost: c_2

Total Cost = $c_1 + c_2$

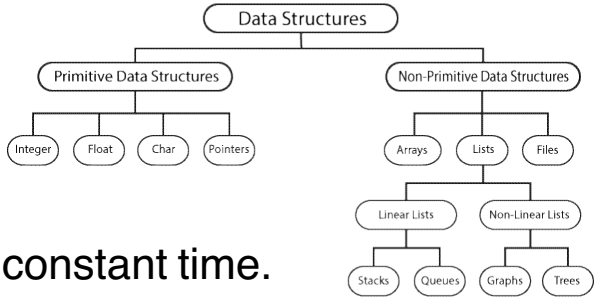
Cost of basic operations

- Challenge. How to estimate constants

operation	example	nanoseconds †
integer add	a + b	2.1
integer multiply	a * b	2.4
integer divide	a / b	5.4
floating-point add	a + b	4.6
floating-point multiply	a * b	4.2
floating-point divide	a / b	13.5
sine	Math.sin(theta)	91.3
arctangent	Math.atan2(y, x)	129.0
...	...	...

† Running OS X on Macbook Pro 2.2GHz with 2GB RAM

Cost of basic operations



- Observation. Most primitive operations take constant time.

operation	example	nanoseconds †
variable declaration	`int a`	c_1
assignment statement	`a = b`	c_2
integer compare	`a < b`	c_3
array element access	`a[i]`	c_4
array length	`a.length`	c_5
1D array allocation	`new int[N]`	$c_6 N$
2D array allocation	`new int[N][N]`	$c_7 N^2$

- Non-primitive operations often take more than constant time.

The Execution Time of Algorithms (cont.)

Example: Simple If-Statement

```
                                Cost        Times
if (n < 0)                      c1          1
   absval = -n                  c2          1
else
     absval = n;                c3          1
```

Total Cost <= c1 + max(c2,c3)

↑
max amount of time ⇒ the condition that will work would def take longer than the idle one

The Execution Time of Algorithms (cont.)

Example: Simple Loop

```
i = 1;                          c1      1
sum = 0;                        c2      1
while (i <= n) {        c3      n+1
      i = i + 1;                c4      n
      sum = sum + i;            c5      n
}
```

so O(n)

Total Cost = c1 + c2 + (n+1)*c3 + n*c4 + n*c5

- The time required for this algorithm is proportional to n

The Execution Time of Algorithms (cont.)

Example: Nested Loop

```
                                        Cost        Times
i=1;                                    c1          1
sum = 0;                                c2          1
while (i <= n) {                        c3          n+1
       j=1;                             c4          n
       while (j <= n) {                 c5          n*(n+1)
           sum = sum + i;               c6          n*n
           j = j + 1;                   c7          n*n
   }
   i = i +1;                            c8          n
}
```

Total Cost = c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5 + n*n*c6 + n*n*c7 + n*c8

- The time required for this algorithm is proportional to n^2

General Rules for Estimation

- **Loops**: The running time of a loop is at most the running time of the statements inside of that loop times the number of iterations.
- **Nested Loops**: Running time of a nested loop containing a statement in the inner most loop is the running time of statement multiplied by the product of the sized of all loops.
- **Consecutive Statements:** Just add the running times of those consecutive statements.
- **If/Else**: Never more than the running time of the test plus the larger of running times of S1 and S2.

Time complexity T(n)

Algorithm 1.2: Add Array Members

- *Problem: Add all the numbers in the array S of n numbers.*
- *Inputs: positive integer n, array of numbers S indexed from 1 to n.*
- *Outputs: sum, the sum of the numbers in S.*

```
number sum (int n, const number S[ ])
{
     index i;
     number result;
     result = 0;
     for (i = 1; i <= n; i++)
        result = result + S[i];
     return result;
}
```

C

n ⇒ O(n)

Regardless of the values of the numbers in the array, there are n passes through the for loop. Therefore, the basic operation is always done n times and

$T(n) = n.$

Time complexity T(n)

Algorithm 1.3: Exchange Sort

- *Problem:Sort n keys in nondecreasing order.*
- *Inputs: positive integer n, array of keys S indexed from 1 to n..*
- *Outputs: the array S containing the keys in nondecreasing order.*

```
void exchangesort (int n, keytype S[])
{
   index i, j;
   for (i=1;i<= n i++)
      for (j=i+1; j<= n; j++)
         if (S[j] < S[i])
                  exchange S[i] and S[j];
}
```

We must determine how many passes there are through the `for-j` loop. For a given n there are always n – 1 passes through the `for-i` loop. In the first pass through the `for-i` loop, there are n – 1 passes through the `for-j` loop, in the second pass there are n – 2 passes through the `for-j` loop, in the third pass there are n - 3 passes through the `for-j` loop, ..., and in the last pass there is one pass through the `for-j` loop. Therefore, the total number of passes through the `for-j` loop is given by

$$T(n) = (n-1) + (n-2) + (n-3) + \cdots + 1 = \frac{(n-1)\,n}{2}.$$

Arithmetic Series

Time complexity T(n)

Algorithm 1.1: Sequential Search

- *Problem: Is the key x in the array S of n keys?*
- *Inputs (parameters): positive integer n, array of keys S indexed from 1 to n, and a key x.*
- *Outputs: location, the location of x in S (0 if x is not in S.)*

```
void seqsearch(int n,
               const keytype S [ ],
               keytype X,
               index & location)

{
     location = 1;
     while (location <= n && S[location] != x)
       location ++;
     if (location > n)
        location=0;
}
```

C

C

n ⇒ O(n)

- **Worst case:** *The basic operation is done at most n times, which is the case if x is the last item in the array or if x is not in the array. Therefore,*

$$W(n) = n.$$

- **Best case:** *Because n ≥ 1, there must be at least one pass through the loop, If x = S[1], there will be one pass through the loop regardless of the size of n. Therefore,*

$$B(n) = 1.$$

Time complexity T(n): Review on Summations

Constant Series: For integers a and b, $a \leq b$, $\sum_{i=a}^{b} 1 = b - a + 1$

Linear Series (Arithmetic Series): For $n \geq 0$, $\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$

Quadratic Series: For $n \geq 0$, $\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$

Time complexity T(n): Review on Summations

Cubic Series: For $n \geq 0$, $\sum_{i=1}^{n} i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$

Geometric Series: For real $x \neq 1$, $\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$

For $|x| < 1$, $\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}$

Time complexity T(n): Common Functions

- **Useful Identities:**

$$a^{-1} = \frac{1}{a}$$

$$(a^m)^n = a^{mn}$$

$$a^m a^n = a^{m+n}$$

- **Exponentials and polynomials**

$$\lim_{n \to \infty} \frac{n^b}{a^n} = 0$$

$$\therefore \; n^b = o(a^n)$$

Time complexity T(n): Common Functions

$x = \log_b a$ is the exponent for $a = b^x$.

Will be mostly used

Natural log: $\ln a = \log_e a$

Binary log: $\lg a = \log_2 a$

$\lg^2 a = (\lg a)^2$

$\lg \lg a = \lg (\lg a)$

$$a = b^{\log_b a}$$

$$\log_c (ab) = \log_c a + \log_c b$$

$$\log_b a^n = n \log_b a$$

$$\log_b (1/a) = -\log_b a$$

$$\log_b a = \frac{1}{\log_a b}$$

$$a^{\log_b c} = c^{\log_b a}$$

Time complexity Order: Big O, Ω, classification

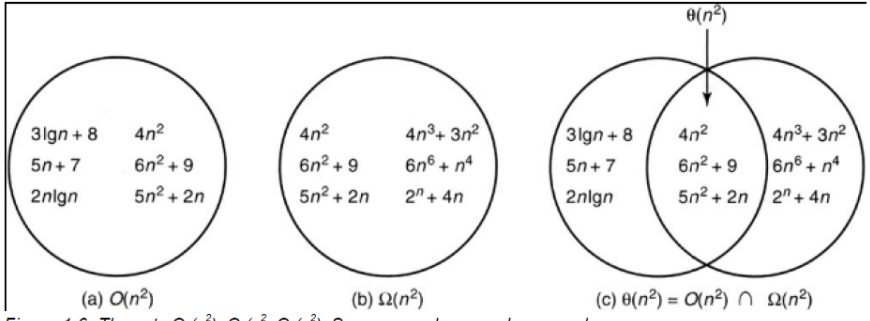


Figure 1.6: The sets O (n^2), O (n^2, O (n^2). Some exemplary members are shown.

Time complexity Order: Big O, Ω, classification

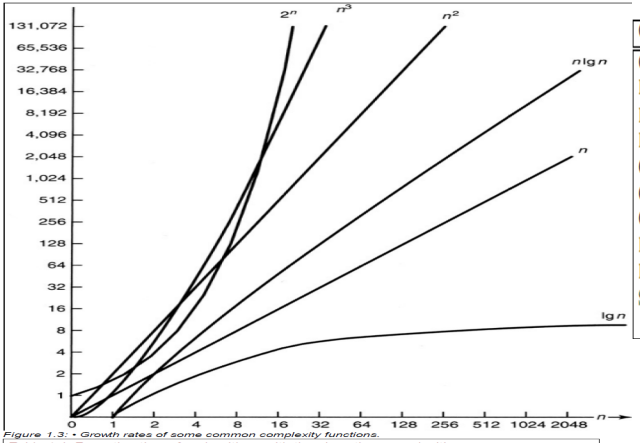


Figure 1.3: • Growth rates of some common complexity functions.

Class Name	Asymptotic Characterization	Algorithm Examples
Constant	$O(1)$	Evaluating a formula
Logarithmic	$O(\log(n))$	Binary Search
Polylogarithmic	$O(\log^k(n))$	
Linear	$O(n)$	Linear Search
Quasilinear	$O(n \log(n))$	Mergesort
Quadratic	$O(n^2)$	Insertion Sort
Cubic	$O(n^3)$	
Polynomial	$O(n^k)$ for any $k > 0$	
Exponential	$O(2^n)$	Computing a powerset
Super-Exponential	$O(2^{f(n)})$ for $f(n) \in \Omega(n)$ For example, $n!$	Computing permutations

Table 1.4: Execution times for algorithms with the given time complexities

n	$f(n) = \lg n$	$f(n) = n$	$f(n) = n \lg n$	$f(n) = n^2$	$f(n) = n^3$	$f(n) = 2^n$
10	0.003 μs^c	0.01 μs	0.033 μs	0.10 μs	1.0 μs	1 μs

Time complexity Order: Big O, Ω, classification

The set of all complexity functions that can be classified with pure quadratic functions is called $\Theta(n^2)$, where Θ is the Greek capital letter "theta." If a function is a member of the set $\Theta(n^2)$, we say that the function is order of n^2. For example, because we can throw away low-order terms,

$$g(n) = 5n^2 + 100n + 20 \in \Theta(n^2),$$

which means that g(n) is order of n^2. As a more concrete example, recall from Section 1.3.1 that the time complexity for Algorithm 1.3 (Exchange Sort) is given by T(n) = n(n – 1)/2. Because

$$\frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2},$$

throwing away the lower-order term n/2 shows that $T(n) \in \Theta(n^2)$.

Time complexity Order: Big O, Ω, classification

When an algorithm's time complexity is in $\Theta(n^2)$, the algorithm is called a ***quadratic-time algorithm*** *or a $\Theta(n^2)$ algorithm. We also say that the algorithm is $\Theta(n^2)$. Exchange Sort is a quadratic-time algorithm.*

Similarly, the set of complexity functions that can be classified with pure cubic functions is called $\Theta(n^3)$, and functions in that set are said to be order of n^3, and so on. We will call these sets ***complexity categories.***
The following are some of the most common complexity categories:

$\Theta(\lg n)$ $\Theta(n)$ $\Theta(n \lg n)$ $\Theta(n^2)$ $\Theta(n^3)$ $\Theta(2^n)$